

🔪 Concept of Pointer

📖 Definition

A **pointer** is a **special variable** that does not store an ordinary value (like a number or character) — instead, it stores the **memory address** of **another variable**. In simple words: **a pointer "points to" the location where another variable lives in memory.**

💡 Why Do We Need Pointers?

Every variable in a C program is stored somewhere in the computer's memory (RAM), and every memory location has a unique **address** — just like every house on a street has a unique address. Normally, we access a variable **by its name** (e.g. x), and C quietly handles the address behind the scenes. A pointer lets us access and work with that **address directly**, which gives us powerful abilities:

- Allows a function to **modify the original variable** of the caller (not just a copy).
- Allows efficient handling of **arrays and strings** (which are closely tied to pointers, as we'll see).
- Enables **dynamic memory allocation** — creating variables/arrays whose size is decided while the program is running.
- Is the foundation for more advanced structures (linked lists, trees) covered in later courses.

🏠 A Simple Real-Life Analogy

👤 Normal Variable

Like a **house** that directly contains a family (the value) living inside it.

🔪 Pointer Variable

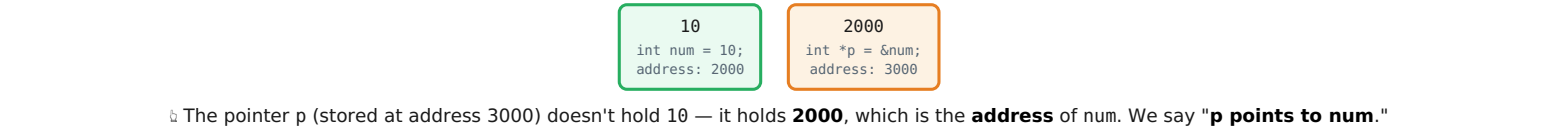
Like a **piece of paper with a house's address written on it**. The paper itself doesn't hold the family — it just tells you **where to find** the house.

🔑 The Two Key Symbols Used With Pointers

Symbol	Name	Meaning	Example
&	Address-of operator	Gives the memory address of a variable	&x → "address of x"
*	Dereference / Value-at operator	Gives the value stored at an address (i.e., "follows" the pointer)	*p → "value at the address p points to"

- ⚠ The symbol * has **two completely different meanings** in C depending on where it's used:
- In a **declaration**: `int *p;` → "p is a pointer to an int" (this * is part of the type)
 - In an **expression**: `*p` → "give me the value at the address p holds" (this * is the dereference operator)

🖥 Visualizing a Pointer



📄 Example Program

```
#include <stdio.h>

int main() {
    int num = 10;
    int *p = &num;    // p now stores the address of num

    printf("Value of num      : %d\n", num);
    printf("Address of num    : %p\n", &num);
    printf("Value stored in p : %p\n", p);
    printf("Value pointed by p : %d\n", *p);

    return 0;
}
```

▶ Output (addresses will vary on your system)

```
Value of num      : 10
Address of num    : 0x7ffee2a3c9ac
Value stored in p : 0x7ffee2a3c9ac
Value pointed by p : 10
```

Notice: "**Address of num**" and "**Value stored in p**" are exactly the **same** — that's the whole idea of a pointer. And *p gives back **10**, because it follows the address stored in p and reads the value sitting there.

🌟 Fun Tip

A pointer is a **"treasure map"** 🗺 — it doesn't hold the treasure, it tells you where to dig!

📌 Declaring & Initializing Pointer Variables

📖 Declaring a Pointer Variable

Declaring a pointer tells the compiler **two things**: the pointer's name, and the **type of variable** it is allowed to point to. This is important because C needs to know the **size** of the data at that address (an `int` and a `double` take different amounts of memory).

🔗 Syntax

```
data_type *pointer_name;
```

🔍 Examples

```
int    *p1;  // pointer to an int
float  *p2;  // pointer to a float
char   *p3;  // pointer to a char
double *p4;  // pointer to a double
```

✦ The `*` can be placed next to the type, next to the name, or with spaces on both sides — all three are treated the **same** by the compiler:

```
int *p;   int* p;   int * p;   // all identical
```

⚠ But be careful with **multiple pointers in one line** — the `*` only applies to the name right after it:

```
int *p1, p2;  // p1 is a pointer to int, but p2 is just a PLAIN int!
int *p1, *p2; // correct way: BOTH p1 and p2 are pointers
```

📖 Initializing a Pointer Variable

Initializing a pointer means giving it a **starting address** to point to. This is usually done using the **address-of operator** `&` on an existing variable.

🔗 Syntax

```
data_type *pointer_name = &variable_name;
```

🔍 Example: Declaration + Initialization Together

```
int age = 25;
int *p = &age;  // p declared AND initialized in one line
```

🔍 Example: Declaration and Initialization Separately

```
int age = 25;
int *p;      // declared first (p currently holds garbage/unknown address)
p = &age;     // initialized separately, later
```

🔑 A Very Important Rule: Type Must Match

The **data type** of the pointer must **match** the data type of the variable it points to. This tells C exactly how many bytes to read/write when using the pointer.

Correct ✓	Incorrect ✗
<code>int a=5; int *p=&a;</code>	<code>int a=5; float *p=&a;</code> (<i>type mismatch</i>)

🔍 Full Example Program

```
#include <stdio.h>

int main() {
    float price = 99.5;
    float *ptr = &price;  // declared and initialized together

    printf("price = %.1f\n", price);
    printf("*ptr   = %.1f\n", *ptr);
    printf("Size of a float pointer = %lu bytes\n", sizeof(ptr));

    return 0;
}
```

▶ Output

```
price = 99.5
*ptr   = 99.5
Size of a float pointer = 8 bytes
```

📌 Note

- Interestingly, **all pointer variables** (whether `int*`, `float*`, or `char*`) take the **same amount of memory** on a given system (commonly 8 bytes on 64-bit systems) — because a pointer only ever stores an **address**, and addresses are always the same size.
- An **uninitialized pointer** is dangerous — it points to a **random, unknown location** in memory (this is called a **wild pointer**, covered on the next page). Always initialize a pointer before using it.

✦ Fun Tip

Declaring a pointer = buying an **empty address book** 📖; initializing = writing a real address in it!

🔑 Accessing Address & Accessing a Variable Through Its Pointer

📖 Part 1: Accessing the Address of a Variable

Every variable, once declared, is automatically given a memory address by the compiler. We can view this address anytime using the **address-of operator &**, placed **before** the variable name.

🔗 Example

```
int x = 50;
printf("Address of x = %p\n", &x);    // %p is the format specifier for addresses
```

▶ Output (example — actual address varies)

```
Address of x = 0x7ffd3a2c1f9c
```

This address is typically shown in **hexadecimal** format and is decided by the operating system when the program runs — it will usually be **different every time** you run the program.

📖 Part 2: Accessing a Variable Through Its Pointer

Once a pointer holds a variable's address, we can access (read) or change (write) that variable's value **indirectly, through the pointer**, using the **dereference operator ***. This is called **indirection**.

🔗 Example Program — Reading AND Modifying Through a Pointer

```
#include <stdio.h>

int main() {
    int x = 50;
    int *p = &x;

    printf("Before: x = %d\n", x);

    *p = 99;    // changes the value AT the address p points to — i.e., changes x itself!

    printf("After:  x = %d\n", x);
    printf("Read through pointer: *p = %d\n", *p);

    return 0;
}
```

▶ Output

```
Before: x = 50
After:  x = 99
Read through pointer: *p = 99
```

This is the **most important idea** in this whole topic: **changing *p actually changes x**, because both are referring to the **exact same memory location**. The pointer is not a copy — it's a direct line to the original variable.

📖 Visualizing "Indirection"



🔑 Quick Reference: Which Symbol Does What

Expression	Meaning	Result (using x=50, p=&x)
x	The value of x directly	50
&x	The address of x	e.g. 5000
p	The value stored in p (which is an address)	5000
*p	The value at the address stored in p	50
&p	The address of the pointer p itself	e.g. 6000

📖 Note

- This ability — to change a variable's value **indirectly** — is exactly why pointers are used to let **functions modify the caller's original variables** (often called "call by reference" / "pass by address").
- %p is the correct format specifier for printing addresses with printf.
- Even the pointer variable p has its **own address** (&p) — a pointer is a variable too, and it lives in memory just like any other variable.

🔑 Fun Tip

& = "Where do you live?" 🏠 | * = "Go there and tell me what's inside!" 🔍

🏠 Special Types of Pointers

📖 Overview

Beyond normal pointers, C has a few **special categories** of pointers that every beginner must recognize — mainly because three of them (**wild**, **dangling**, and uninitialized pointers) are **common sources of bugs**, while the other two (**NULL** and **generic/void**) are **safe, useful tools**.

1 NULL Pointer

A **NULL pointer** is a pointer that has been **deliberately set to point to nothing**. It is C's way of saying "*this pointer is currently not pointing to any valid variable*."

```
int *p = NULL;    // p intentionally points to nothing (address 0)
```

✔ **Safe practice:** Always check if a pointer is NULL before dereferencing it, especially if it was returned from a function that might fail (like memory allocation):

```
if (p != NULL) {
    printf("%d", *p);    // safe to dereference
} else {
    printf("Pointer is empty!\n");
}
```

2 Wild Pointer

A **wild pointer** is a pointer that has been **declared but never initialized** — it holds a **random, unpredictable address** left over in memory (garbage value).

```
int *p;           // WILD — p contains a garbage/random address
printf("%d", *p); // DANGEROUS — could crash, or corrupt unrelated memory
```

⚠ **Danger:** Dereferencing a wild pointer leads to **undefined behaviour** — the program might crash, or worse, silently corrupt other data without any warning.

Fix: Always initialize a pointer — either to a real address, or to NULL — the moment you declare it.

3 Dangling Pointer

A **dangling pointer** is a pointer that was **once valid**, but now points to a memory location that has been **freed, deleted, or gone out of scope** — the address it holds is no longer "owned" by that data.

🔗 Example: Dangling Pointer

```
int *getPointer() {
    int val = 100;    // local variable — exists only inside this function
    return &val;      // returning its address is a mistake!
} // val is destroyed the moment the function ends

int main() {
    int *p = getPointer(); // p is now DANGLING — points to destroyed memory
    printf("%d", *p);      // undefined behaviour!
    return 0;
}
```

Difference from wild: a wild pointer was **never valid**; a dangling pointer **used to be valid** but the data it pointed to is now gone.

4 Generic Pointer (void pointer)

A **generic pointer** (declared using void*) is a **special pointer that can point to any data type** — int, float, char, or anything else. It is called "generic" because it isn't tied to one specific type.

```
void *ptr;
int a = 10;
float b = 5.5;

ptr = &a;    // valid — void* can point to an int
ptr = &b;    // also valid — void* can point to a float too
```

- A void* pointer **cannot be directly dereferenced** — it must first be **typecast** to the correct type: *(int*)ptr or *(float*)ptr.
- Generic pointers are commonly used in library functions that need to work with **any** data type (e.g. malloc() returns a void*).

📊 Summary Table

Type	Points to	Safe to use?
NULL Pointer	Nothing (intentionally, address 0)	✔ Safe — used to represent "empty" deliberately
Wild Pointer	Random, unknown address (never set)	✗ Unsafe — never dereference
Dangling Pointer	An address that is no longer valid (data destroyed)	✗ Unsafe — never dereference
Generic (void) Pointer	Any data type — type not fixed	✔ Safe, but must typecast before dereferencing

💡 Fun Tip

NULL = "empty, on purpose" ✔ | Wild = "never told where to go" 😬 | Dangling = "house got demolished" 🏠

Definition

A **pointer expression** is any valid combination of **pointers, operators, and variables** that C can evaluate — just like a normal expression, but involving **addresses** instead of (or along with) plain values.

Common Kinds of Pointer Expressions

Expression Type	Example	Meaning
Assignment	<code>p = &x;</code>	Store address of <code>x</code> into pointer <code>p</code>
Dereferencing	<code>y = *p;</code>	Copy the value pointed to by <code>p</code> into <code>y</code>
Pointer Arithmetic	<code>p + 1</code>	Address of the next element (covered in detail next page)
Comparison	<code>p1 == p2</code>	Checks if two pointers hold the same address
Assigning one pointer to another	<code>p2 = p1;</code>	Now both pointers point to the same location
Combined with arithmetic	<code>*p + 5</code>	Take the value pointed to by <code>p</code> , then add 5 to it

Understanding Operator Precedence in Pointer Expressions

Since `*` can combine with other operators, precedence matters a lot. Compare these two carefully:

Expression	Meaning
<code>*p + 1</code>	Dereference <code>p</code> first (get the value), then add 1 to that value
<code>*(p + 1)</code>	Add 1 to the address <code>p</code> first (move to next element), then dereference it

These two expressions look almost identical but do **completely different things** — this small difference is one of the biggest sources of confusion for beginners, so always double-check with parentheses when combining `*` with arithmetic.

Example Program

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30};
    int *p = arr; // p points to arr[0]

    printf("*p + 1   = %d\n", *p + 1); // value at arr[0] (10), plus 1   = 11
    printf("*(p + 1) = %d\n", *(p + 1)); // value at arr[1] (20)           = 20

    int *q = p; // q now points to the SAME location as p
    printf("p == q ? %s\n", (p == q) ? "Yes" : "No");

    return 0;
}
```

Output

```
*p + 1   = 11
*(p + 1) = 20
p == q ? Yes
```

Note

- Pointers can be **compared** using `==`, `!=`, `<`, `>` — this is mainly meaningful when both pointers point **into the same array** (to check relative position).
- You **cannot multiply or divide** two pointers, or add two pointers together — only **addition/subtraction with an integer**, and **subtracting one pointer from another** (of the same array), are meaningful (see Pointer Arithmetic).
- Always use **parentheses** to make your intention clear when mixing `*` with `+`, `-`, etc.

Fun Tip

`*p + 1` vs `*(p+1)` → same characters, totally different meaning – always check your brackets! 🔍

+ Pointer Arithmetic

Definition

Pointer arithmetic refers to the specific set of mathematical operations that are allowed on pointers — mainly used to **move a pointer from one array element to the next**. Pointer arithmetic is "**smart**": it automatically accounts for the **size of the data type** the pointer points to.

The Golden Rule of Pointer Arithmetic

When you write `p + 1`, C does **not** simply add 1 to the numeric address. Instead, it adds **1 × size_of(data_type)**. This is what makes pointers so useful for navigating arrays.

`p + 1` → actually moves the address forward by: `sizeof(data_type)` bytes

Pointer Type	sizeof(type)	<code>p + 1</code> moves forward by
<code>char *p</code>	1 byte	1 byte
<code>int *p</code>	4 bytes (typical)	4 bytes
<code>double *p</code>	8 bytes (typical)	8 bytes

Visualizing Pointer Arithmetic on an int Array

For `int arr[4] = {10, 20, 30, 40};` where each `int` takes 4 bytes:

10	20	30	40
<code>arr[0]</code>	<code>arr[1]</code>	<code>arr[2]</code>	<code>arr[3]</code>
1000	1004	1008	1012

Expression	Address	Value (<code>*expression</code>)
<code>p (= arr)</code>	1000	10
<code>p + 1</code>	1004 <i>(jumped by 4, not 1!)</i>	20
<code>p + 2</code>	1008	30
<code>p + 3</code>	1012	40

Operations Allowed on Pointers

Operation	Allowed?	Meaning
<code>pointer + integer</code>	✔ Yes	Move forward by that many elements
<code>pointer - integer</code>	✔ Yes	Move backward by that many elements
<code>pointer++ / pointer--</code>	✔ Yes	Move to next / previous element
<code>pointer1 - pointer2</code>	✔ Yes (same array)	Gives the number of elements between them
<code>pointer1 + pointer2</code>	✗ No	Meaningless — adding two addresses makes no sense
<code>pointer * / integer</code>	✗ No	Multiplication/division of pointers is not allowed

Example Program — Traversing an Array Using Pointer Arithmetic

```
#include <stdio.h>

int main() {
    int arr[4] = {10, 20, 30, 40};
    int *p = arr;    // p points to arr[0]

    for (int i = 0; i < 4; i++) {
        printf("Address: %p    Value: %d\n", (void*)(p + i), *(p + i));
    }

    int *last = &arr[3];
    printf("Elements between p and last = %ld\n", last - p);

    return 0;
}
```

► Output (addresses will vary)

```
Address: 0x...1000    Value: 10
Address: 0x...1004    Value: 20
Address: 0x...1008    Value: 30
Address: 0x...100c    Value: 40
Elements between p and last = 3
```

Note

- Pointer arithmetic only makes sense when the pointer is moving **within the same array** — moving beyond its bounds gives **undefined behaviour** (same risk as array out-of-bounds access).
- `p++` is extremely common when looping through arrays using pointers instead of index numbers.
- This "automatic scaling by data type size" is **exactly why** the pointer's declared type must match the data it points to.

Fun Tip

`p + 1` = "next locker down the row" ➡️, not "next byte"!

🔗 Pointer Representation of Array

📖 The Key Idea

In C, **arrays and pointers are deeply connected**. The name of an array, when used in an expression, automatically represents the **address of its first element**. This means:

```
arr // is the same as &arr[0]
```

This is why an array name can be assigned **directly** to a pointer, without needing &:

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr; // valid! same as: int *p = &arr[0];
```

🔗 Two Equivalent Ways to Access Array Elements

Because of this connection, **every** array element can be accessed in **two equivalent ways** — using normal index notation, or using pointer notation:

Index Notation	Pointer Notation	Meaning
arr[0]	*(arr + 0) or *p	First element
arr[1]	*(arr + 1) or *(p + 1)	Second element
arr[i]	*(arr + i) or *(p + i)	i-th element

In fact, the compiler **internally converts** arr[i] into *(arr + i) anyway — the square-bracket notation is just a more readable **shortcut** for pointer arithmetic!

🔗 Example Program — Same Output, Two Different Notations

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;

    for (int i = 0; i < 5; i++) {
        printf("arr[%d] = %d    |    *(p+%d) = %d\n", i, arr[i], i, *(p + i));
    }

    return 0;
}
```

▶ Output

```
arr[0] = 10    |    *(p+0) = 10
arr[1] = 20    |    *(p+1) = 20
arr[2] = 30    |    *(p+2) = 30
arr[3] = 40    |    *(p+3) = 40
arr[4] = 50    |    *(p+4) = 50
```

🔗 One Important Difference: arr vs p

- ⚠️ **Careful:** although arr "acts like" a pointer, it is **not exactly a pointer variable**:
- arr is a **constant address** — you **cannot** do arr = arr + 1; or arr++; (this would try to change where the array itself starts, which isn't allowed).
 - p (a real pointer variable) **can** be changed: p = p + 1; or p++; is perfectly valid, because p is just a normal variable that happens to store an address.

🔗 Note

- When an array is **passed to a function**, what actually gets passed is its **address** (a pointer to the first element) — this is why functions can modify the original array's contents.
- This array-pointer relationship applies to **1-D arrays**; multi-dimensional arrays involve "pointers to arrays" which is a more advanced extension of the same idea.

🔗 Fun Tip

An array name is like a **signboard nailed to the ground** 🚧 — it always points to the same first house, and can't be moved!

📦 Array of Pointers

📖 Definition

An **array of pointers** is exactly what it sounds like — an **array whose elements are themselves pointers**, instead of ordinary values. Each "box" in this array doesn't hold a number or character — it holds an **address**.

🔧 Syntax

```
data_type *array_name[size];
```

🔍 Read this from the **name outward**: `array_name` is an **array** of **size** elements, and each element is a **pointer to data_type**.

🔍 Example — Array of Pointers to Integers

```
#include <stdio.h>

int main() {
    int a = 10, b = 20, c = 30;

    int *ptrArr[3]; // array of 3 pointers to int

    ptrArr[0] = &a;
    ptrArr[1] = &b;
    ptrArr[2] = &c;

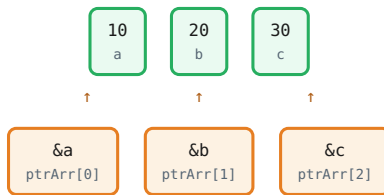
    for (int i = 0; i < 3; i++) {
        printf("*ptrArr[%d] = %d\n", i, *ptrArr[i]);
    }

    return 0;
}
```

▶ Output

```
*ptrArr[0] = 10
*ptrArr[1] = 20
*ptrArr[2] = 30
```

🖼 Visualizing an Array of Pointers



🔑 The Most Common Real Use: Array of Strings

The **most practical use** of an array of pointers is to store a **list of strings**, since (as we know) a string is essentially handled through a char pointer:

🔍 Example — Array of Pointers to Strings

```
#include <stdio.h>

int main() {
    char *days[5] = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday"};

    for (int i = 0; i < 5; i++) {
        printf("Day %d: %s\n", i+1, days[i]);
    }

    return 0;
}
```

▶ Output

```
Day 1: Monday
Day 2: Tuesday
Day 3: Wednesday
Day 4: Thursday
Day 5: Friday
```

Here, `days` is an array of **5 character pointers**, and each pointer holds the **address of the first character** of its respective word — this is far more memory-efficient than a fixed 2-D character array, since each word can be a **different length**.

📌 Note

- An array of pointers is different from a **pointer to an array** (e.g. `int (*p)[5];`) — the parentheses matter a lot; that is a more advanced topic covered separately.
- Storing strings as an array of pointers is more flexible than a 2-D char array, because each string can be a **different length** without wasting memory.

💡 Fun Tip

Array of pointers = a **row of treasure maps** 🗺️ — each one points to a different treasure!

📖 Accessing a String Using Pointer

📖 Recap: What Is a String, Again?

As we saw earlier, a string in C is a **character array ending in '\0'**. Since an array name already behaves like a pointer to its first element, a string can **naturally** be accessed and processed using a char pointer.

🔗 Two Ways to Represent a String

Method	Example	Nature
Character array	<code>char str[] = "Hello";</code>	Memory allocated for the array; content can be modified
Character pointer	<code>char *str = "Hello";</code>	<code>str</code> just points to a string literal stored elsewhere; content should not be modified

🔗 Example — Accessing a String Character-by-Character Using a Pointer

```
#include <stdio.h>

int main() {
    char *str = "Hello";

    while (*str != '\0') {    // keep going until we hit the null character
        printf("%c", *str);  // print current character
        str++;               // move pointer to next character
    }
    printf("\n");

    return 0;
}
```

▶ Output

Hello

🔗 Step-by-Step: How the Loop Walks Through the String

1. `str` starts by pointing to 'H'. Condition true → print **H** → `str++` moves to 'e'.
2. This repeats for 'e', 'l', 'l', 'o'.
3. After printing 'o', `str++` moves to '\0'. Condition `*str != '\0'` becomes **false** → loop stops.

🔗 Writing Our Own `strlen()` Using a Pointer

This is a classic exercise that shows exactly how string functions work internally:

```
#include <stdio.h>

int myStrLen(char *s) {
    int count = 0;
    while (*s != '\0') {
        count++;
        s++;
    }
    return count;
}

int main() {
    char *name = "Programming";
    printf("Length = %d\n", myStrLen(name));
    return 0;
}
```

▶ Output

Length = 11

📖 Character Array vs Character Pointer — Key Differences

Aspect	<code>char str[] = "Hi";</code>	<code>char *str = "Hi";</code>
Memory	A new, separate copy of "Hi" is created	<code>str</code> points to a fixed string literal (often stored in read-only memory)
Can modify contents?	✔ Yes — <code>str[0]='J';</code> is safe	✘ Risky/undefined — modifying string literals can crash the program
Can reassign to a new string?	✘ No — array's storage is fixed	✔ Yes — <code>str = "Bye";</code> is valid, since <code>str</code> is just a pointer

📖 Note

- Passing a string to a function (e.g. `myStrLen(name)`) actually passes a **character pointer** — this is why string-handling functions are written using `char *` parameters.
- Always loop **until '\0'** when processing a string manually with a pointer — forgetting this check is a common bug that reads past the end of the string.
- Standard library functions like `strlen`, `strcpy`, and `strcat` (seen earlier) are all internally implemented using exactly this kind of pointer-walking logic.

🔗 Fun Tip

Walking a string with a pointer = following a trail of footprints 🐾 until you hit the "THE END" sign '\0'!